ELE432: Digital Design and Computer Architecture
Lab 3: Multicycle Processor - Preliminary Work Report

Name: Alperen SALMAN **Date:** April 28, 2026

Subject: Implementation and Simulation of a Multicycle RISC-V Processor

1. Lab Duration

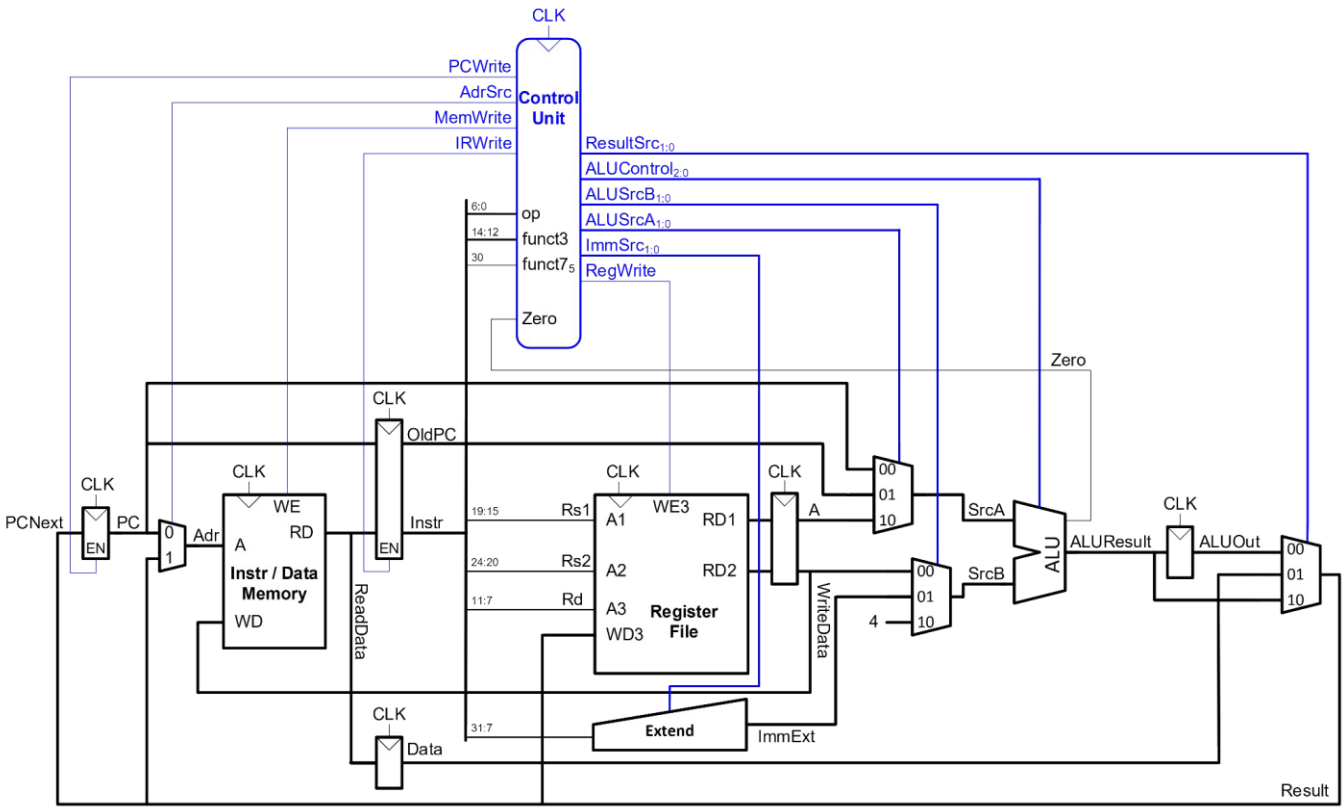
Hours Spent: 8 Hours

2. Processor Hierarchy & Block Diagram

The processor is built using a modular approach to separate the control logic from the data processing units.

- **top**: Connects the RISC-V core to the unified memory .
- **riscv**: The core CPU containing the controller and datapath.
- **controller**: Manages the FSM states and decodes ALU instructions.
- **datapath**: Contains the ALU, Register File, and instruction/data registers.

Processor Block Diagram



3. Hierarchical SystemVerilog Description

The following modules were implemented to create the multicycle architecture.

1) **top.sv**:

```
module top (
   input  logic       clk, reset,
   output logic [31:0] WriteData, DataAdr,
   output logic       MemWrite
);

   logic [31:0] ReadData;

   // Instantiate the RISC-V multicycle processor core
   riscv rv (clk, reset, DataAdr, WriteData, MemWrite, ReadData);

   // Instantiate the unified memory (Instruction + Data)
   mem m (clk, MemWrite, DataAdr, WriteData, ReadData);

endmodule
```

2) **mem.sv**:

```
module mem (
   input  logic       clk, we,
   input  logic [31:0] a, wd,
   output logic [31:0] rd
);

   logic [31:0] RAM[63:0];

   // Load the machine code file
   initial
      $readmemh("riscvtest.txt", RAM);

   // Read logic: Word aligned (ignores bottom two bits)
   assign rd = RAM[a[31:2]];

   // Synchronous write logic
   always_ff @(posedge clk)
      if (we) RAM[a[31:2]] <= wd;

endmodule
```

3) **riscv.sv**:

```
module riscv (
   input  logic       clk, reset,
   output logic [31:0] Adr, WriteData,
   output logic       MemWrite,
   input  logic [31:0] ReadData
);

   // Internal Control Wires between Controller and Datapath
   logic [1:0]  ResultSrc, ALUSrcA, ALUSrcB;
   logic [2:0]  ALUControl, ImmSrc;
   logic       RegWrite, PCWrite, AdrSrc, IRWrite;
   logic       Zero;
   logic [31:0] Instr;

   // Instantiate the Controller (FSM)
   controller c (
      clk, reset,
      Instr[6:0], Instr[14:12], Instr[30], // op, funct3, funct7
      Zero,
      ImmSrc, ALUSrcA, ALUSrcB, ResultSrc,
      AdrSrc, ALUControl, IRWrite, PCWrite, RegWrite, MemWrite
   );

   // Instantiate the Datapath (Hardware blocks)
   datapath dp (
      clk, reset,
      ImmSrc, ALUSrcA, ALUSrcB, ResultSrc,
      AdrSrc, ALUControl, IRWrite, PCWrite, RegWrite,
      Zero, Instr, Adr, WriteData, ReadData
   );
endmodule
```

4) **controller.sv**:

```
module controller(
   input  logic      clk, reset,
   input  logic [6:0] op,
   input  logic [2:0] funct3,
   input  logic      funct7b5,
   input  logic      Zero,
   output logic [2:0] ImmSrc,
   output logic [1:0] ALUSrcA, ALUSrcB,
   output logic [1:0] ResultSrc,
   output logic      AdrSrc,
   output logic [2:0] ALUControl,
   output logic      IRWrite, PCWrite, RegWrite, MemWrite
);

   logic [1:0] ALUOp;
   logic      Branch, PCUpdate;
```

```
    // Main Decoder FSM (Using NAMED mapping for safety)
    mainfsm fsm(
        .clk(clk), .reset(reset), .op(op),
        .ImmSrc(ImmSrc), .ALUSrcA(ALUSrcA), .ALUSrcB(ALUSrcB), .ResultSrc(ResultSrc),
        .AdrSrc(AdrSrc), .IRWrite(IRWrite), .PCUpdate(PCUpdate),
        .RegWrite(RegWrite), .MemWrite(MemWrite), .Branch(Branch), .ALUOp(ALUOp)
    );

    // ALU Decoder
    aludec ad(op[5], funct3, funct7b5, ALUOp, ALUControl);

    // Branch logic for PCUpdate
    assign PCWrite = (Branch & Zero) | PCUpdate;

endmodule
```

5) **mainfsm.sv**:

```
module mainfsm(
    input  logic      clk, reset,
    input  logic [6:0] op,
    output logic [2:0] ImmSrc,
    output logic [1:0] ALUSrcA, ALUSrcB, ResultSrc,
    output logic      AdrSrc, IRWrite, PCUpdate, RegWrite, MemWrite, Branch,
    output logic [1:0] ALUOp
);

    // State definitions based on Table 1 and RISC-V multicycle flow
    typedef enum logic [3:0] {
        S0_FETCH, S1_DECODE, S2_MEMADR, S3_MEMREAD, S4_MEMWB, S5_MEMWRITE,
        S6_EXECUTER, S7_ALUWB, S8_EXECUTEI, S9_JAL, S10_BEQ
    } statetype;

    statetype state, nextstate;

    // --- State Register ---
    always_ff @(posedge clk or posedge reset)
        if (reset) state <= S0_FETCH;
        else       state <= nextstate;

    // --- Next State Logic ---
    always_comb
        case (state)
            S0_FETCH:   nextstate = S1_DECODE;
            S1_DECODE:  case(op)
                        7'b0000011: nextstate = S2_MEMADR;   // lw
                        7'b0100011: nextstate = S2_MEMADR;   // sw
                        7'b0110011: nextstate = S6_EXECUTER; // R-type
                        7'b0010011: nextstate = S8_EXECUTEI; // I-type (addi)
                        7'b1101111: nextstate = S9_JAL;      // jal
                        7'b1100011: nextstate = S10_BEQ;     // beq
                        default:    nextstate = S0_FETCH;
                    endcase
```

```
        S2_MEMADR:  if (op == 7'b0000011) nextstate = S3_MEMREAD;
                  else                nextstate = S5_MEMWRITE;
        S3_MEMREAD:  nextstate = S4_MEMWB;
        S4_MEMWB:    nextstate = S0_FETCH;
        S5_MEMWRITE: nextstate = S0_FETCH;
        S6_EXECUTER: nextstate = S7_ALUWB;
       S7_ALUWB:    nextstate = S0_FETCH;
        S8_EXECUTEI: nextstate = S7_ALUWB;
        S9_JAL:      nextstate = S7_ALUWB;
        S10_BEQ:     nextstate = S0_FETCH;
        default:     nextstate = S0_FETCH;
    endcase

// --- Output Logic (Control Signals) ---
// Signals mapped according to Figure 1
always_comb begin
    // Default values to ensure no latches are created
    {AdrSrc, IRWrite, RegWrite, MemWrite, PCUpdate, Branch} = 6'b0;
    {ALUSrcA, ALUSrcB, ResultSrc, ALUOp} = 8'b0;

    case (state)
        S0_FETCH: begin
            AdrSrc = 1'b0;     // Select PC as address
            IRWrite = 1'b1;    // Enable Instruction Register write
            ALUSrcA = 2'b00;   // SrcA = PC
            ALUSrcB = 2'b10;   // SrcB = 4
            ALUOp = 2'b00;     // Add
            ResultSrc = 2'b10; // Result = ALUResult
            PCUpdate = 1'b1;   // Trigger PC update
        end
        S1_DECODE: begin
            ALUSrcA = 2'b01;   // SrcA = OldPC
            ALUSrcB = 2'b01;   // SrcB = ImmExt
            ALUOp = 2'b00;     // Add (pre-calculate branch target)
        End
        S2_MEMADR: begin
            ALUSrcA = 2'b10;   // SrcA = Reg RD1
            ALUSrcB = 2'b01;   // SrcB = ImmExt
            ALUOp = 2'b00;     // Add (calculate memory address)
        end
        S3_MEMREAD: begin
            AdrSrc = 1'b1;     // Select ALUOut as address
            ResultSrc = 2'b00;
        end
        S4_MEMWB: begin
            ResultSrc = 2'b01; // Result = Data from memory
            RegWrite = 1'b1;   // Write to register file
        end
        S5_MEMWRITE: begin
            AdrSrc = 1'b1;     // Select ALUOut as address
            MemWrite = 1'b1;   // Enable memory write
            ResultSrc = 2'b00;
        end
```

```
        S6_EXECUTER: begin
          ALUSrcA = 2'b10;   // SrcA = Reg RD1
          ALUSrcB = 2'b00;   // SrcB = Reg RD2
          ALUOp = 2'b10;     // Logic defined by funct3/7
        end
        S7_ALUWB: begin
          ResultSrc = 2'b00; // Result = ALUOut
          RegWrite = 1'b1;   // Write to register file
        end
        S8_EXECUTEI: begin
          ALUSrcA = 2'b10;   // SrcA = Reg RD1
          ALUSrcB = 2'b01;   // SrcB = ImmExt
          ALUOp = 2'b10;     // Logic defined by funct3
        end
        S9_JAL: begin
          ALUSrcA = 2'b01;   // SrcA = OldPC
          ALUSrcB = 2'b10;   // SrcB = 4
          ALUOp = 2'b00;     // Add
          ResultSrc = 2'b00; // Result = ALUOut
          PCUpdate = 1'b1;   // Jump update
        end
        S10_BEQ: begin
          ALUSrcA = 2'b10;   // SrcA = Reg RD1
          ALUSrcB = 2'b00;   // SrcB = Reg RD2
          ALUOp = 2'b01;     // Subtract (for Zero comparison)
          ResultSrc = 2'b00;
          Branch = 1'b1;     // Enable branch logic
        end
      endcase
    end

    // --- Immediate Decoder ---
    // Logic for ImmSrc based on instruction type
    always_comb
      case(op)
        7'b0000011: ImmSrc = 3'b000; // I-type (lw)
        7'b0010011: ImmSrc = 3'b000; // I-type (addi)
        7'b0100011: ImmSrc = 3'b001; // S-type (sw)
        7'b0110011: ImmSrc = 3'bxxx; // R-type
        7'b1100011: ImmSrc = 3'b010; // B-type (beq)
        7'b1101111: ImmSrc = 3'b011; // J-type (jal)
        default:    ImmSrc = 3'b000;
      endcase

endmodule
```

6) **aludec.sv**:

```
module aludec(
   input  logic      opb5,
   input  logic [2:0] funct3,
   input  logic      funct7b5,
   input  logic [1:0] ALUOp,
   output logic [2:0] ALUControl
);

   logic  RtypeSub;
   assign RtypeSub = funct7b5 & opb5; // TRUE for SUB instruction

   always_comb
      case(ALUOp)
         2'b00: ALUControl = 3'b000; // addition
         2'b01: ALUControl = 3'b001; // subtraction
         default: case(funct3) // R-type or I-type ALU
                  3'b000: if (RtypeSub) ALUControl = 3'b001; // sub
                        else        ALUControl = 3'b000; // add
                  3'b010: ALUControl = 3'b101; // slt
                  3'b110: ALUControl = 3'b011; // or
                  3'b111: ALUControl = 3'b010; // and
                  default: ALUControl = 3'bxxx; // unknown
                 endcase
      endcase
endmodule
```

7) **datapath.sv**:

```
module datapath (
   input  logic       clk, reset,
   input  logic [2:0]  ImmSrc,
   input  logic [1:0]  ALUSrcA, ALUSrcB,
   input  logic [1:0]  ResultSrc,
   input  logic       AdrSrc,
   input  logic [2:0]  ALUControl,
   input  logic       IRWrite, PCWrite, RegWrite,
   output logic       Zero,
   output logic [31:0] Instr,
   output logic [31:0] Adr, WriteData,
   input  logic [31:0] ReadData
);

   logic [31:0] PC, OldPC;
   logic [31:0] Data, RD1, RD2, A, SrcA, SrcB;
   logic [31:0] ALUResult, ALUOut, Result, ImmExt;
```

```
    // --- Memory and Instruction Logic ---

    // PC Register: Updates only when PCWrite is high
    flopenr #(32) pcreg(clk, reset, PCWrite, Result, PC);
    // Address Mux: Selects between PC (Fetch) and Result (Mem Access)
    mux2 #(32)   adrmux(PC, Result, AdrSrc, Adr);

    // Instruction Register: Captures new instruction from memory
    flopenr #(32) ir(clk, reset, IRWrite, ReadData, Instr);
    flopenr #(32) oldpcreg(clk, reset, IRWrite, PC, OldPC);

    // Data Register: Buffer for data read from memory
    flopr #(32)  datareg(clk, reset, ReadData, Data);

     // --- Register File and Extend Logic ---

    regfile      rf(clk, RegWrite, Instr[19:15], Instr[24:20],
                 Instr[11:7], Result, RD1, RD2);

    extend       ext(Instr[31:7], ImmSrc, ImmExt);

    // Pipelines registers to hold values for the Execute stage
    flopr #(32)  areg(clk, reset, RD1, A);
    flopr #(32)  wdreg(clk, reset, RD2, WriteData);

    // --- ALU Logic ---

    // ALUSrcA Mux: Selects PC, OldPC, or Register A
    mux3 #(32)   srcamux(PC, OldPC, A, ALUSrcA, SrcA);

    // ALUSrcB Mux: Selects Register B (WriteData), ImmExt, or constant 4
    mux3 #(32)   srcbmux(WriteData, ImmExt, 32'd4, ALUSrcB, SrcB);

    alu         alu_inst(SrcA, SrcB, ALUControl, ALUResult, Zero);

    // ALUOut register: Stores ALU result for use in a later cycle
    flopr #(32)  aluoutreg(clk, reset, ALUResult, ALUOut);

    // --- Result Logic ---

    // ResultSrc Mux: Final value to be written to PC or RegFile
    mux3 #(32)   resmux(ALUOut, Data, ALUResult, ResultSrc, Result);

endmodule
```

8) **buildingblocks.sv**:

```
// 2-way Multiplexer
module mux2 #(parameter WIDTH = 8) (
   input  logic [WIDTH-1:0] d0, d1,
   input  logic           s,
   output logic [WIDTH-1:0] y
);
   assign y = s ? d1 : d0;
endmodule

// 3-way Multiplexer
module mux3 #(parameter WIDTH = 8) (
   input  logic [WIDTH-1:0] d0, d1, d2,
   input  logic [1:0]      s,
   output logic [WIDTH-1:0] y
);
   assign y = (s == 2'b00) ? d0 :
            (s == 2'b01) ? d1 : d2;
endmodule

// Resettable Flip-Flop
module flopr #(parameter WIDTH = 8) (
   input  logic           clk, reset,
   input  logic [WIDTH-1:0] d,
   output logic [WIDTH-1:0] q
);
   always_ff @(posedge clk, posedge reset)
      if (reset) q <= 0;
      else       q <= d;
endmodule

// Resettable Flip-Flop with Enable
module flopenr #(parameter WIDTH = 8) (
   input  logic           clk, reset, en,
   input  logic [WIDTH-1:0] d,
   output logic [WIDTH-1:0] q
);
   always_ff @(posedge clk, posedge reset)
      if (reset)   q <= 0;
      else if (en) q <= d;
endmodule
```

9) **regfile.sv**:

```
module regfile(
   input  logic       clk,
   input  logic       we3,
   input  logic [4:0]  a1, a2, a3,
   input  logic [31:0] wd3,
   output logic [31:0] rd1, rd2
);

   logic [31:0] rf[31:0];

   // Three-ported register file
   // Read two ports combinationally (A1/RD1, A2/RD2)
   // Write third port on rising edge of clock (A3/WD3/WE3)
   // Register 0 hardwired to 0

   always_ff @(posedge clk)
      if (we3) rf[a3] <= wd3;

   assign rd1 = (a1 != 0) ? rf[a1] : 0;
   assign rd2 = (a2 != 0) ? rf[a2] : 0;
endmodule
```

10) **alu.sv**:

```
module alu(
   input  logic [31:0] a, b,
   input  logic [2:0]  alucontrol,
   output logic [31:0] result,
   output logic       zero
);

   logic [31:0] condinvb, sum;
   logic       v;             // overflow
   logic       isAddSub;      // true of addition or subtraction

   assign condinvb = alucontrol[0] ? ~b : b;
   assign sum = a + condinvb + alucontrol[0];
   assign isAddSub = ~alucontrol[2] & ~alucontrol[1] |
                     ~alucontrol[2] & alucontrol[0];

   always_comb
      case (alucontrol)
         3'b000:  result = sum;            // add
         3'b001:  result = sum;            // subtract
         3'b010:  result = a & b;          // and
         3'b011:  result = a | b;          // or
         3'b101:  result = sum[31];        // slt
         default: result = 32'bx;
      endcase

   assign zero = (result == 32'b0);
endmodule
```

11) **extend.sv**:

```
module extend(
   input  logic [31:7] instr,
   input  logic [2:0]  immsrc,
   output logic [31:0] immext
);

   always_comb
      case(immsrc)

         3'b000:  immext = {{20{instr[31]}}, instr[31:20]}; // I-type (addi, lw)
         3'b001:  immext = {{20{instr[31]}}, instr[31:25], instr[11:7]}; // S-type (sw)
         3'b010:  immext = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}; // B-type (beq)
         3'b011:  immext = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0}; // J-type (jal)
         default: immext = 32'bx;
      endcase
endmodule
```

4. Table 1: Expected Operation

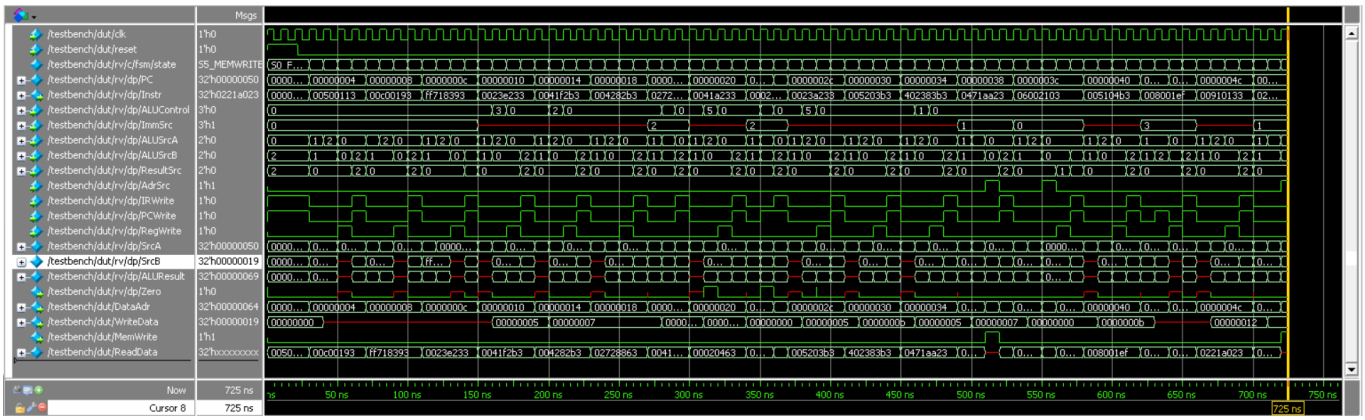
This table tracks the first three instructions of the test program (addi operations).

Step	PC	Instr	State	Result	Result Notes
1	00	00500113	S0: Fetch	4	ALU calculates PC + 4
2	04	00500113	S1: Decode	X	ALU calculates OldPC + Imm
3	04	00500113	S8: ExecuteI	5	ALUResult = 0 + 5
4	04	00500113	S7: ALUWB	5	Register x2 written with 5
5	04	00C00193	S0: Fetch	8	ALU calculates PC + 4
6	08	00C00193	S1: Decode	X	ALU calculates OldPC + Imm
7	08	00C00193	S8: ExecuteI	12	ALUResult = 0 + 12
8	08	00C00193	S7: ALUWB	12	Register x3 written with 12
9	08	FF718393	S0: Fetch	12	ALU calculates PC + 4
10	0C	FF718393	S1: Decode	X	ALU calculates OldPC + Imm
11	0C	FF718393	S8: ExecuteI	3	ALUResult = 12 - 9
12	0C	FF718393	S7: ALUWB	3	Register x7 written with 3

5. Simulation Waveforms

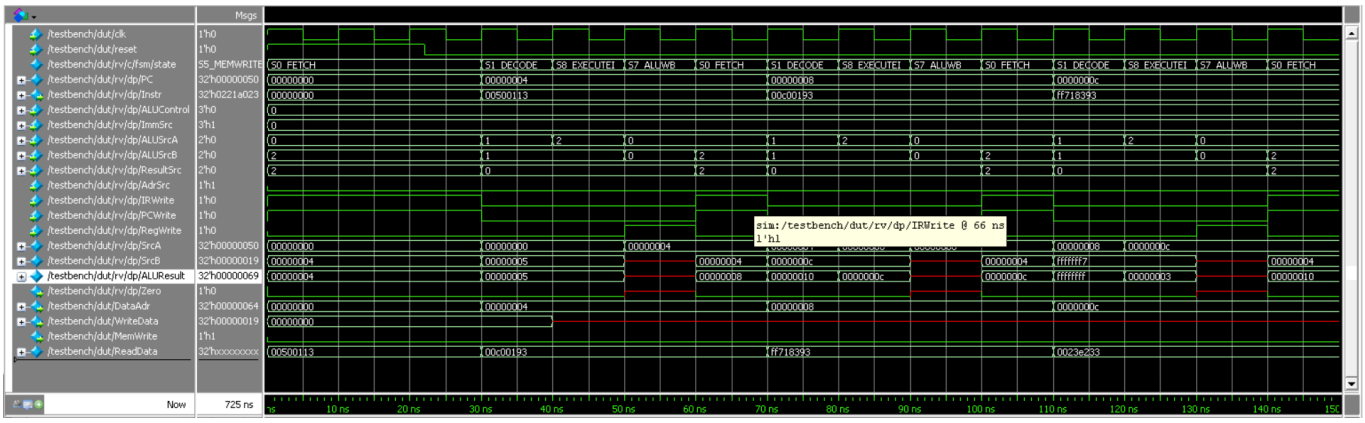
The simulation was performed in Questa to verify the timing and functional accuracy of the control signals .

Full Simulation Overview:



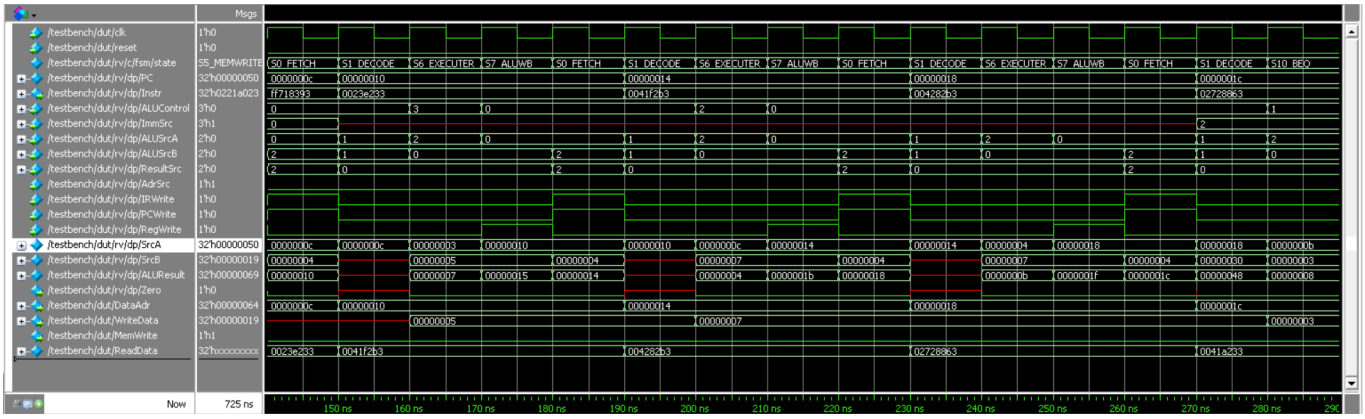
Partial Simulation Overviews:

1. **0ns - 150ns:**



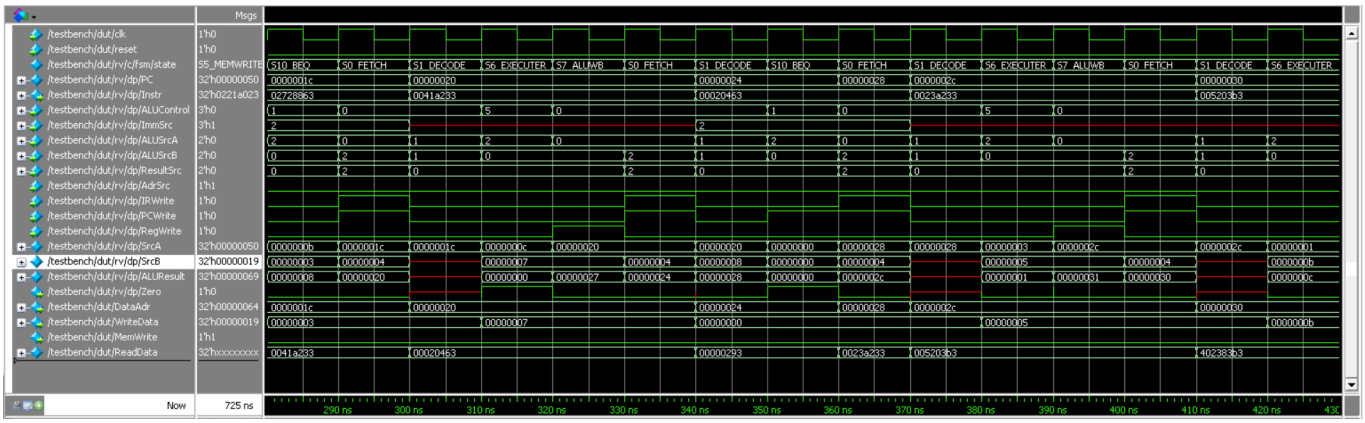
Simulation result in 0ns - 150ns

2. **140ns - 290ns**



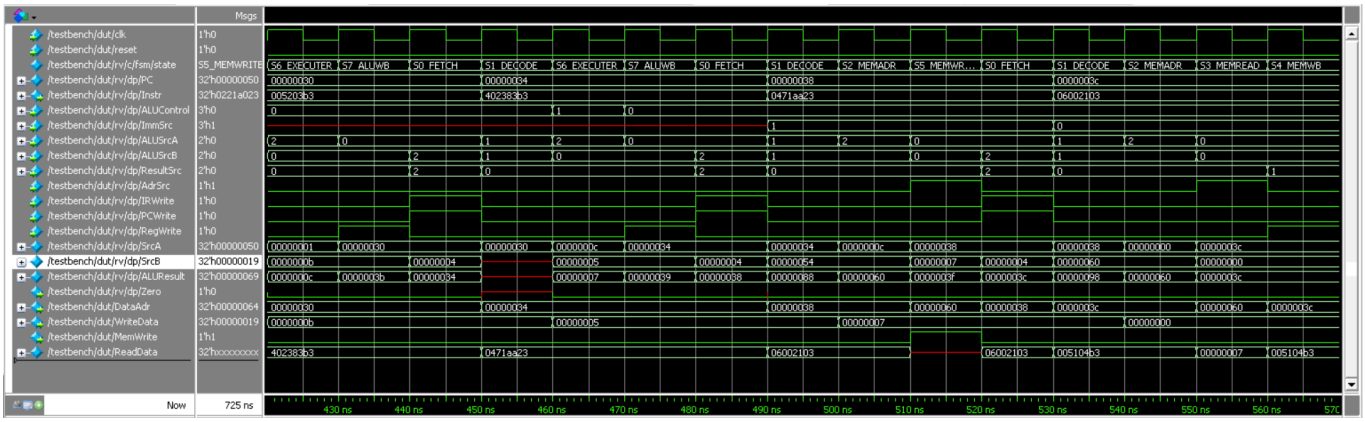
Simulation result in 140ns - 290ns

3. 280ns - 430ns:



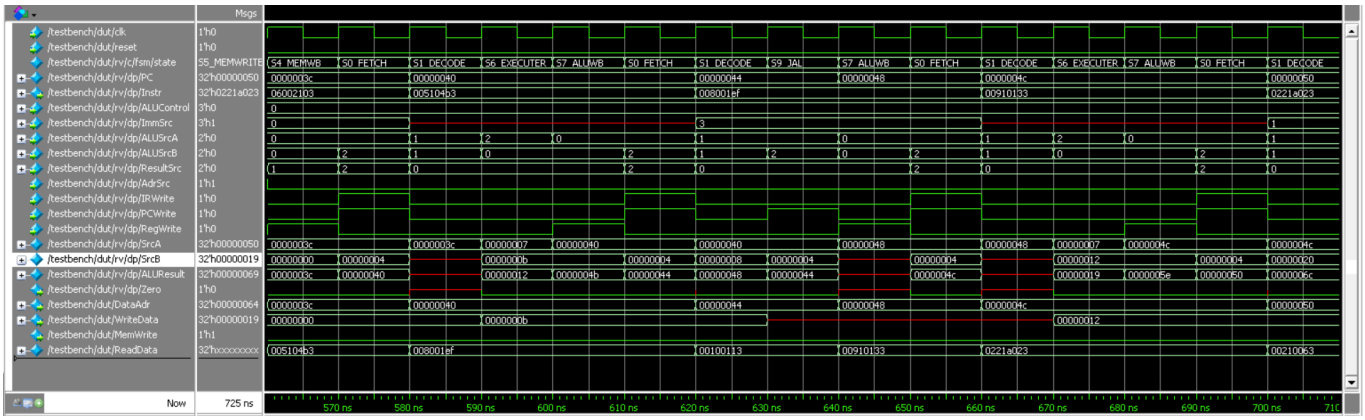
Simulation result in 280ns - 430ns

4. 420ns - 570ns:



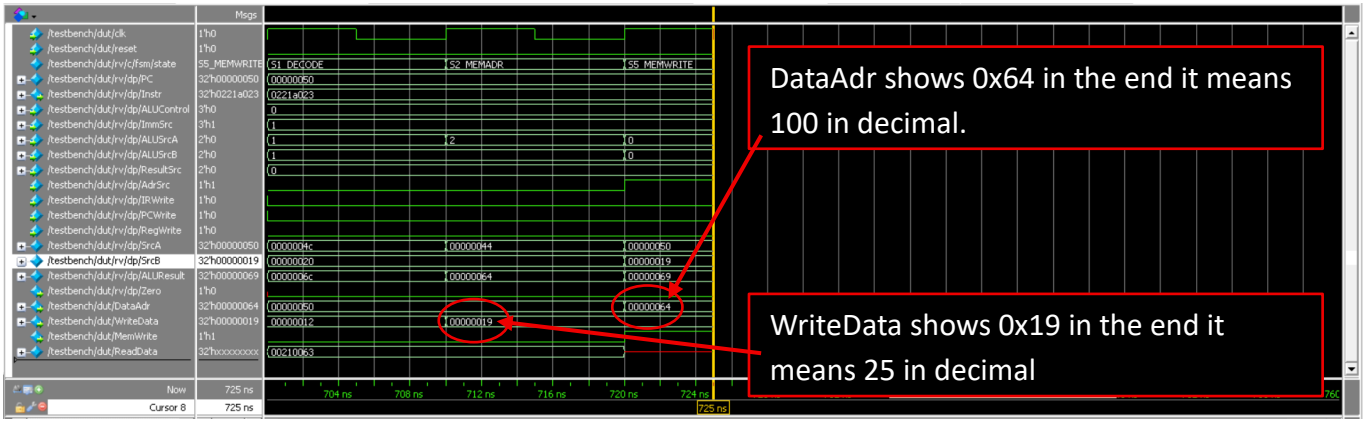
Simulation result in 420ns - 570ns

5. 560ns - 710ns:



Simulation result in 560ns - 710ns

6. **700ns - 725ns:**



Simulation result in 700ns - 725ns

Transcript Console Output

```
Transcript
VSIM 40> run -all
# Simulation succeeded
# ** Note: $stop    : C:/Users/Alperen SALMAN/AppData/Local/quartus/Lab3/riscv_testbench.sv(30)
#    Time: 725 ns  Iteration: 1  Instance: /testbench
# Break in Module testbench at C:/Users/Alperen SALMAN/AppData/Local/quartus/Lab3/riscv_testbench.sv line 30
```

- **Description:** The Questa transcript confirms the successful completion of the test program. The simulation reached the $stop command at line 30 of riscv_testbench.sv exactly at **725 ns**, printing the message **"Simulation succeeded"**. This indicates that the processor successfully navigated all R-type, I-type, and Branch instructions to reach the final memory write.

6. Conclusion & Debugging

The multicycle RISC-V processor successfully passed the verification suite .

- **Logic Verification**: The testbench reported **"Simulation succeeded"** at **725 ns**, confirming that all instructions (R-type, I-type, Load/Store, and Branch) are functioning as intended .
- **Debugging**: A critical race condition was found in the OldPC logic. By converting the OldPC register to a flopenr and using IRWrite as an enable, the processor now correctly maintains the current instruction's address for jal return calculations.
- **Final Result**: The final memory write to address **100** with value **25** proves the overall architectural integrity of the design .

Git Repository Link

URL: https://github.com/Alperen-Salman/ELE432_Lab3_prework